6 · Exp 7 — Insertion sort (and its performance)

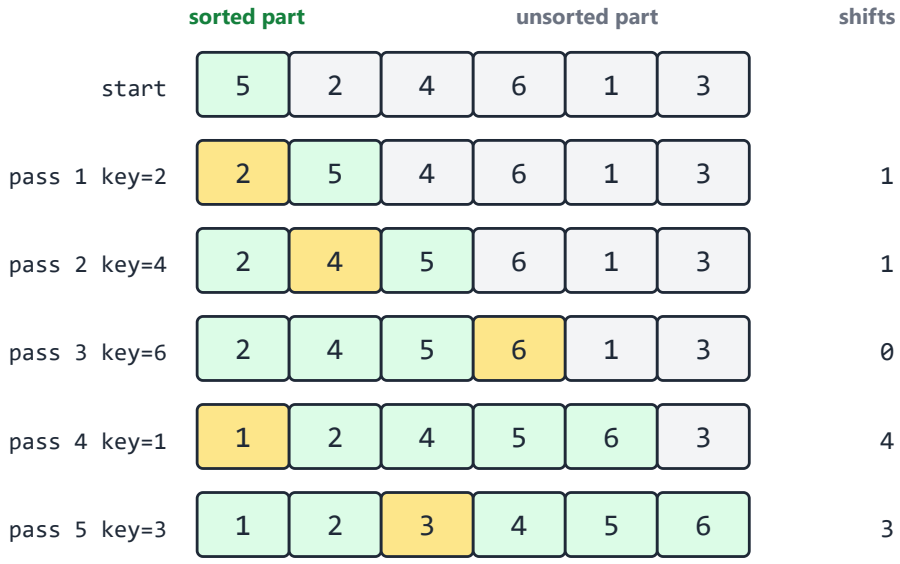
You already know the core move from Exp 3's "insert into a sorted array". Insertion sort just does it for every element in turn.

Sorted part / unsorted part → Pick key → **Shift bigger ones right** → Drop key in gap → Repeat

6.1 The idea

Like sorting playing cards in your hand: the left part is already sorted. Pick up the next card (the **key**), slide every bigger card one place right, and drop the key into the gap.

> **GUESS FIRST**
>
> `5 2 4 6 1 3`. How many single-place shifts does insertion sort need in total? (Count them pass by pass.) And how many for the already-sorted `1 2 3 4 5 6`?



yellow = where the key landed; total shifts = 9

The manual's example, pass by pass. Green = sorted part; yellow = where the key landed.

```
int key = arr[i];
int j = i - 1;
while (j >= 0 && arr[j] > key)   // bigger than key? move it right
{
    arr[j + 1] = arr[j];
    j--;
}
arr[j + 1] = key;                // j stopped one left of the gap
```

WHY `ARR[J + 1] = KEY` AND NOT `ARR[J]`

The loop stops when `arr[j]` is **not** bigger than the key (or j = −1). The gap is one to the right of that element, at j + 1.

Why `j >= 0` comes **first**: when the key is the smallest (key = 1 in pass 4), j reaches −1. `&&` stops at the first false, so `arr[-1]` is never read. Swap the order and the program reads outside the array.

6.2 The program (corrected)

The manual's sort was correct. The only fix: `int arr[n]` (a size read at run time) isn't standard C++, and strict compilers reject it:

G++ SAYS

```
m07_vla.cpp:25:9: error: ISO C++ forbids variable length array 'arr' [-Wvla]
   25 |     int arr[n];
      |         ^~~
```

So the array is `int arr[100]` with a range check on n.

E07_INSERTION_SORT.CPP

```cpp
#include <iostream>
using namespace std;

// Function to perform Insertion Sort
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];                    // card picked up
        int j = i - 1;

        // Shift elements greater than key one position to the right
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;                    // drop the card in the gap
    }
}

// Function to display the array
void displayArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main()
{
    int n;
    int arr[100];                            // FIX: was int arr[n] (not standard C++)
```

```
    cout << "Enter number of elements: ";
    cin >> n;

    if (n < 1 || n > 100)
    {
        cout << "n must be between 1 and 100.";
        return 0;
    }

    cout << "Enter elements of the array:" << endl;
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Original Array: ";
    displayArray(arr, n);

    insertionSort(arr, n);

    cout << "Sorted Array: ";
    displayArray(arr, n);

    return 0;
}
```

SCREEN (REAL RUN)

```
Enter number of elements: 6
Enter elements of the array:
5 2 4 6 1 3
Original Array: 5 2 4 6 1 3
Sorted Array: 1 2 3 4 5 6
```

Duplicates and negatives work too (the strict `>` keeps equal elements in their original order, which is why the sort is **stable**):

SCREEN (REAL RUN)

```
Enter number of elements: 7
Enter elements of the array:
3 -1 3 0 9 -1 2
Original Array: 3 -1 3 0 9 -1 2
Sorted Array: -1 -1 0 2 3 3 9
```

6.3 Performance analysis (the "analyse" half of the aim)

These counts come from the tested counting version (section 6.5):

Input (n = 6)	Comparisons	Shifts	Case
1 2 3 4 5 6 (sorted)	5 = n − 1	0	**best**: O(n)
5 2 4 6 1 3 (manual)	12	9	average: $O(n^2)$
6 5 4 3 2 1 (reversed)	15 = n(n−1)/2	15	**worst**: $O(n^2)$

- **Best case O(n):** each key is compared once with its left neighbour, and stops.
- **Worst case O(n^2):** key i is compared with and shifted past all i elements before it: 1 + 2 + … + (n−1) = n(n−1)/2.
- **Space O(1):** sorts inside the same array using only `key` and `j`. It's **in-place**.

WRITE IN YOUR RECORD

Aim: To sort an integer array in ascending order using insertion sort and analyse its performance.

Algorithm: for i = 1 to n−1: key = arr[i], j = i−1; while j ≥ 0 and arr[j] > key: arr[j+1] = arr[j], j−−; then arr[j+1] = key. Print the array.

Complexity: best O(n) (sorted input), average and worst O(n^2) (reverse-sorted input: n(n−1)/2 comparisons); space O(1).

Result: 5 2 4 6 1 3 → 1 2 3 4 5 6 in 5 passes (9 shifts). Insertion sort is in-place, stable, and fast for small or nearly sorted data.

6.4 Viva questions

Question	Answer
Why does the outer loop start at i = 1?	A single element (index 0) is already "sorted".
Best, average, worst case?	O(n), O(n^2), O(n^2).
What input gives the best case? The worst?	Already sorted; reverse sorted.
Is it stable? In-place?	Both yes: `>` (not `>=`) keeps equal keys in order; only O(1) extra memory.
When is insertion sort a good choice?	Small arrays, nearly-sorted data, or data arriving one item at a time ("online").
Why save arr[i] in key?	The first shift overwrites arr[i]; without the copy the value would be lost.
How many passes for n elements?	n − 1.

6.5 If the examiner changes the question

"Separate functions for input, sorting and display" is the program above (add an `inputArray` function if asked). The rest:

Descending order (one character changes):

```
// Descending: the ONLY change is  arr[j] > key  ->  arr[j] < key
void insertionSortDesc(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] < key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Count comparisons and shifts. Used for the table in 6.3, and covers "compare performance for sorted, reverse-sorted and random arrays": run it on all three and print the counts.

```
// Counts comparisons (each arr[j] > key test) and shifts (each arr[j+1] = arr[j])
void insertionSortCount(int arr[], int n, int &comparisons, int &shifts)
{
    comparisons = 0;
    shifts = 0;
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0)
        {
            comparisons++;
            if (arr[j] > key)
            {
                arr[j + 1] = arr[j];
                shifts++;
                j--;
            }
            else
                break;
        }
        arr[j + 1] = key;
    }
}
```

Call: `int c, s; insertionSortCount(arr, n, c, s); cout << "Comparisons = " << c << ", Shifts = " << s;`

Floating-point numbers:

```c
// Floating-point version: change the type of the array and of key - nothing else
void insertionSortFloat(float arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        float key = arr[i];
        int j = i - 1;                      // j is an index: stays int
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Position of an element after sorting:

```c
// After sorting, find where x ended up (1-based position, -1 if absent)
int findPosition(int arr[], int n, int x)
{
    for (int i = 0; i < n; i++)
        if (arr[i] == x)
            return i + 1;
    return -1;
}
```

CHECK 6.1

Sort `4 3 2 10 12 1 5 6` by hand, pass by pass. Write the array after each pass.

CHECK 6.2

Change one character in the while condition to sort in **descending** order. Which character, and what does it become?

Answers

Check 6.1: key 3 → `3 4 2 10 12 1 5 6`; key 2 → `2 3 4 10 12 1 5 6`; key 10 → no change; key 12 → no change; key 1 → `1 2 3 4 10 12 5 6`; key 5 → `1 2 3 4 5 10 12 6`; key 6 → `1 2 3 4 5 6 10 12`. Seven passes (n − 1).

Check 6.2: `arr[j] > key` → `arr[j] < key`: move **smaller** elements right, so bigger ones end up first.